

Coverage Metric - GPU Optimized

Karina Arichavala

2026-04-21

Table of contents

0.1	1. Setup e Imports	1
0.2	2. Cargar Datos	2
0.3	3. Cargar SAE y Extraer Features	3
0.4	4. Filtrar BSPs de Piezas	4
0.5	5. Implementación de Coverage	5
0.5.1	Estrategia de optimización:	5
0.6	6. Calcular Coverage	7
0.7	7. Análisis de Resultados	8
0.8	8. Guardar Resultados	13
0.9	9. Generar PDF con Quarto	13

```
# Configuración para visualización en PDF
import pandas as pd
import numpy as np
import sys

# Para que el texto de pandas no se corte
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 100) # Ancho fijo para pandas

# Para prints largos de numpy - ancho más conservador
np.set_printoptions(linewidth=100, edgeitems=3)

# Configurar ancho de terminal para prints
import os
os.environ['COLUMNS'] = '100'

print(" Configuración para PDF lista")
```

Configuración para PDF lista

0.1 1. Setup e Imports

```
import numpy as np
import torch
import sys
import os
from pathlib import Path
from tqdm import tqdm
import time

# Configurar paths
project_root = Path('../..').resolve()
sys.path.insert(0, str(project_root))
```

```

from sae.models.sae import SparseAutoencoder
from sae.tools.naming_utils import get_checkpoint_dir, get_model_name, get_extras_id
from sae.tools.experiment_utils import get_experiment_dir, get_metrics_dir, get_report_name

# Verificar GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memoria disponible: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

```

Device: cuda
GPU: NVIDIA GeForce RTX 5060
Memoria disponible: 8.55 GB

```

#Metadatataparanamingdecheckpoints(separadadehiperparámetros)
NAMING_META={
    'variante': 'standard', #ArquitecturadelSAE
    'tecnica': 'l1-decay', #L1conpenaltyannealing
    'capa': 6, #LayerdelmodeloOthelloGPT
    'juegos':200, #Númerodepartidas
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt', #Pathbaseparacheckpoints
    'experiment_base_path':os.path.abspath('../../../experiments'), #sae/experiments/
    'extras':{
        'expansion':16,
        'sparsity_coef':5e-4,
        'epochs':30,
        'patience': 5
    }
}

print('\n Metadata del experimento:')
for key, value in NAMING_META.items():
    print(f'  {key}: {value}')

```

Metadata del experimento:
variante: standard
tecnica: l1-decay
capa: 6
juegos: 200
base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\experiments
extras: {'expansion': 16, 'sparsity_coef': 0.0005, 'epochs': 30, 'patience': 5}

0.2 2. Cargar Datos

```

# Cargar activaciones pre-SAE
activations_path = project_root / "sae" / "activations" / "data" / "layer6_1games.npy"
activations_full = np.load(activations_path)

# Filtrar solo los últimos 20 tableros de la partida
activations = activations_full[-15:]

print(f"Activaciones del modelo (últimos 15 tableros):")
print(f"  Shape original: {activations_full.shape}")
print(f"  Shape final: {activations.shape}")
print(f"  Memoria: {activations.nbytes / (1024**2):.2f} MB")

```

```
Activaciones del modelo (últimos 15 tableros):  
  Shape original: (59, 512)  
  Shape final: (15, 512)  
  Memoria: 0.03 MB
```

```
# Cargar ground truth de BSPs  
bsp_gt_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_1games.npy"  
bsp_names_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_1games.names.npy"  
  
bsp_ground_truth_full = np.load(bsp_gt_path)  
bsp_names = np.load(bsp_names_path, allow_pickle=True)  
  
# Filtrar solo los últimos 15 tableros (alineado con las activaciones)  
bsp_ground_truth = bsp_ground_truth_full[-15:]  
  
print(f"\nGround truth BSPs (últimos 15 tableros):")  
print(f"  Shape original: {bsp_ground_truth_full.shape}")  
print(f"  Shape final: {bsp_ground_truth.shape}")  
print(f"  Total BSPs: {len(bsp_names)}")
```

```
Ground truth BSPs (últimos 15 tableros):  
  Shape original: (59, 326)  
  Shape final: (15, 326)  
  Total BSPs: 326
```

0.3 3. Cargar SAE y Extraer Features

```
# Configuración del SAE  
input_dim = 512  
hidden_dim = 8192  
  
# Construir ruta del modelo usando naming_utils  
checkpoint_dir = get_checkpoint_dir(  
    NAMING_META['base_path'],  
    NAMING_META['variante'],  
    NAMING_META['tecnica'],  
    NAMING_META['capa'],  
    NAMING_META['juegos']  
)  
extras_id = get_extras_id(checkpoint_dir, NAMING_META['extras']) if NAMING_META.get('extras') else None  
  
model_name = get_model_name(  
    NAMING_META['variante'],  
    NAMING_META['tecnica'],  
    NAMING_META['capa'],  
    NAMING_META['juegos'],  
    'best',  
    extras_id=extras_id  
)  
model_path = checkpoint_dir / model_name  
  
# Cargar modelo (weights_only=False: checkpoint contiene objetos Path en config)  
sae = SparseAutoencoder(input_dim, hidden_dim).to(device)  
checkpoint = torch.load(model_path, map_location=device, weights_only=False)  
sae.load_state_dict(checkpoint['model_state_dict'])  
sae.eval()  
  
print(f"SAE cargado:")  
print(f"  Path: {model_path}")
```

```
print(f"  Input: {input_dim}, Hidden: {hidden_dim}")
print(f"  Expansion: {hidden_dim/input_dim}x")
print(f"  Epoch: {checkpoint.get('epoch', 'N/A')}, Val MSE: {checkpoint.get('val_mse', float('nan')):.6f}")
```

SAE cargado:

```
Path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_standard\sae_l1-decay_standard\200games\sae
decay_l6_200g_ex1_best.pt
Input: 512, Hidden: 8192
Expansion: 16.0x
Epoch: N/A, Val MSE: nan
```

```
# Extraer features del SAE en GPU
print("Extrayendo features del SAE...")
activations_tensor = torch.from_numpy(activations).float().to(device)

with torch.no_grad():
    sae_features = torch.relu(sae.encoder(activations_tensor))

print(f"\nFeatures SAE:")
print(f"  Shape: {sae_features.shape}")
print(f"  Device: {sae_features.device}")
print(f"  Sparsity: {(sae_features == 0).float().mean():.2%}")
print(f"  Activaciones promedio: {(sae_features > 0).sum(dim=1).float().mean():.1f}")
```

Extrayendo features del SAE...

Features SAE:

```
Shape: torch.Size([15, 8192])
Device: cuda:0
Sparsity: 89.11%
Activaciones promedio: 891.9
```

0.4 4. Filtrar BSPs de Piezas

Coverage solo usa las 128 BSPs de piezas (más/oponente), sin vacías.

```
# =====
# CONFIGURACIÓN DE FILTRADO DE BSPs
# =====
USE_PLAYER   = False # BSPs perspectiva del jugador (1 y 2)
USE_ABSOLUTE = True  # BSPs absolutas negro/blanco (B y W)
USE_STRATEGIC = False # BSPs especiales (BSP_)

# Filtrar según configuración
bsp_piezas_indices = []
bsp_piezas_names = []

for i, name in enumerate(bsp_names):
    include = False

    if USE_PLAYER and len(name) == 6 and name.startswith('BSP') and (name.endswith('1') or name.endswith('2')):
        include = True
    if USE_ABSOLUTE and (name.endswith('B') or name.endswith('W')):
        include = True
    if USE_STRATEGIC and name.startswith('BSP_'):
        include = True

    if include:
        bsp_piezas_indices.append(i)
        bsp_piezas_names.append(name)
```

```

bsp_pieces_indices = np.array(bsp_pieces_indices)
bsp_pieces_gt = bsp_ground_truth[:, bsp_pieces_indices]

# Convertir a tensor en GPU
bsp_pieces_gt_tensor = torch.from_numpy(bsp_pieces_gt).bool().to(device)

print(f"BSPs seleccionadas para Coverage:")
print(f"  Player: {USE_PLAYER} | Absolute: {USE_ABSOLUTE} | Strategic: {USE_STRATEGIC}")
print(f"  Total: {len(bsp_pieces_indices)}")
print(f"  Shape: {bsp_pieces_gt_tensor.shape}")
print(f"  Device: {bsp_pieces_gt_tensor.device}")
print(f"  Primeras 5: {'', '.join(bsp_pieces_names[:5])}")
print(f"  Últimas 5: {'', '.join(bsp_pieces_names[-5:])}")
# Identificador del filtrado (para naming de archivos de salida)
parts = []
if USE_PLAYER: parts.append("player")
if USE_ABSOLUTE: parts.append("absolute")
if USE_STRATEGIC: parts.append("strategic")
filter_suffix = "_".join(parts) if parts else "none"
print(f"  Filter suffix: {filter_suffix}")

```

```

BSPs seleccionadas para Coverage:
  Player: False | Absolute: True | Strategic: False
  Total: 128
  Shape: torch.Size([15, 128])
  Device: cuda:0
  Primeras 5: BSPA1B, BSPA1W, BSPA2B, BSPA2W, BSPA3B
  Últimas 5: BSPH6W, BSPH7B, BSPH7W, BSPH8B, BSPH8W
  Filter suffix: absolute

```

0.5 5. Implementación de Coverage

0.5.1 Estrategia de optimización:

1. **Vectorización:** Procesar múltiples features en paralelo usando operaciones matriciales
2. **Batch processing:** Dividir en chunks para no saturar memoria GPU
3. **Pre-cálculo:** Calcular máximos una sola vez
4. **Early stopping:** Terminar cuando F1 = 1.0

```

def fast_f1_score_gpu(y_true, y_pred, eps=1e-8):
    """
    F1 score vectorizado en GPU.

    Args:
        y_true: Tensor (n_positions,) booleano
        y_pred: Tensor (n_positions, n_candidates) booleano

    Returns:
        f1_scores: Tensor (n_candidates,)
    """
    # y_true: (n_positions,) -> expand to (n_positions, 1)
    y_true_expanded = y_true.unsqueeze(1) # (n_positions, 1)

    # Calcular TP, FP, FN
    tp = (y_true_expanded & y_pred).sum(dim=0).float() # (n_candidates,)
    fp = (~y_true_expanded & y_pred).sum(dim=0).float()
    fn = (y_true_expanded & ~y_pred).sum(dim=0).float()

    # Precision y Recall
    precision = tp / (tp + fp + eps)

```

```

recall = tp / (tp + fn + eps)

# F1
f1 = 2 * (precision * recall) / (precision + recall + eps)

return f1

def calculate_coverage_gpu_optimized(
    sae_features, # Tensor (n_positions, n_features) en GPU
    bsp_ground_truth, # Tensor (n_positions, n_bsps) en GPU, bool
    thresholds=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9],
    batch_size_features=512, # Procesar 512 features a la vez
    verbose=True
):
    """
    Calcula Coverage de forma optimizada para GPU.

    Procesa múltiples features y thresholds en paralelo usando operaciones matriciales.
    """
    n_positions, n_features = sae_features.shape
    n_bsps = bsp_ground_truth.shape[1]
    n_thresholds = len(thresholds)

    if verbose:
        print("="*60)
        print("CALCULATING COVERAGE (GPU OPTIMIZED)")
        print("="*60)
        print(f"Positions: {n_positions:,}")
        print(f"SAE Features: {n_features:,}")
        print(f"BSPs: {n_bsps:,}")
        print(f"Thresholds: {n_thresholds:,}")
        print(f"Batch size (features): {batch_size_features:,}")
        print(f"Total evaluations: {n_bsps * n_features * n_thresholds:,}")
        print("="*60)

    # Pre-calcular máximos de features
    f_max = sae_features.max(dim=0)[0] # (n_features,)
    active_features = (f_max > 0).nonzero(as_tuple=True)[0]
    n_active = len(active_features)

    if verbose:
        print(f"\nActive features: {n_active:,}/{n_features:,} ({n_active/n_features*100:.1f}%)")
        print(f"Reduced evaluations: {n_bsps * n_active * n_thresholds:,}")
        print()

    # Almacenar resultados
    best_f1s = torch.zeros(n_bsps, device=sae_features.device)
    best_features = torch.full((n_bsps,), -1, dtype=torch.long, device=sae_features.device)
    best_thresholds = torch.full((n_bsps,), -1.0, device=sae_features.device)

    thresholds_tensor = torch.tensor(thresholds, device=sae_features.device)

    # Procesar cada BSP
    pbar = tqdm(range(n_bsps), desc="Processing BSPs") if verbose else range(n_bsps)

    for bsp_idx in pbar:
        bsp_labels = bsp_ground_truth[:, bsp_idx] # (n_positions,)

        # Skip si la BSP nunca está activa
        if not bsp_labels.any():
            continue

```

```

# Procesar features activas en batches
for batch_start in range(0, n_active, batch_size_features):
    batch_end = min(batch_start + batch_size_features, n_active)
    batch_features_idx = active_features[batch_start:batch_end]

    # Extraer features del batch
    batch_activations = sae_features[:, batch_features_idx] # (n_positions, batch_size)
    batch_f_max = f_max[batch_features_idx] # (batch_size,)

    # Para cada threshold, binarizar TODAS las features del batch a la vez
    for t in thresholds_tensor:
        # Binarizar: (n_positions, batch_size)
        predictions = batch_activations > (t * batch_f_max.unsqueeze(0))

        # Calcular F1 para todas las features del batch
        f1_scores = fast_f1_score_gpu(bsp_labels, predictions) # (batch_size,)

        # Encontrar la mejor en este batch
        max_f1, max_idx_in_batch = f1_scores.max(dim=0)

        # Actualizar si es mejor que lo visto hasta ahora
        if max_f1 > best_f1s[bsp_idx]:
            best_f1s[bsp_idx] = max_f1
            best_features[bsp_idx] = batch_features_idx[max_idx_in_batch]
            best_thresholds[bsp_idx] = t

    # Early stopping: si ya es casi perfecto, no revisar más batches
    if best_f1s[bsp_idx] >= 0.999:
        break

# Calcular Coverage (macro-average)
coverage = best_f1s.mean().item()

# Convertir a CPU para retornar
best_f1s_cpu = best_f1s.cpu().numpy()
best_features_cpu = best_features.cpu().numpy()
best_thresholds_cpu = best_thresholds.cpu().numpy()

return coverage, best_f1s_cpu, best_features_cpu, best_thresholds_cpu

print(" Funciones de Coverage GPU definidas")

```

Funciones de Coverage GPU definidas

0.6 6. Calcular Coverage

```

# Medir tiempo de ejecución
start_time = time.time()

coverage, best_f1s, best_features, best_thresholds = calculate_coverage_gpu_optimized(
    sae_features,
    bsp_pieces_gt_tensor,
    batch_size_features=512, # Ajustar según memoria GPU
    verbose=True
)

elapsed_time = time.time() - start_time

print("\n" + "="*60)

```

```

print("RESULTADO COVERAGE")
print("="*60)
print(f"Coverage Score: {coverage:.4f}")
print(f"Objetivo (paper): 0.52")
print(f"Diferencia: {coverage - 0.52:+.4f}")
print(f"\nTiempo de ejecución: {elapsed_time:.2f} seg")
print(f"                                ({elapsed_time/60:.2f} min)")
print("="*60)

=====
CALCULATING COVERAGE (GPU OPTIMIZED)
=====

Positions: 15
SAE Features: 8,192
BSPs: 128
Thresholds: 10
Batch size (features): 512
Total evaluations: 10,485,760
=====

Active features: 3,832/8,192 (46.8%)
Reduced evaluations: 4,904,960

Processing BSPs: 100%|      | 128/128 [00:02<00:00, 44.62it/s]

=====
RESULTADO COVERAGE
=====

Coverage Score: 0.7182
Objetivo (paper): 0.52
Diferencia: +0.1982

Tiempo de ejecución: 2.93 seg
                    (0.05 min)
=====

```

0.7 7. Análisis de Resultados

```

# Estadísticas de distribución de F1 scores
print("Distribución de F1 scores:")
print(f"  Mínimo: {best_f1s.min():.4f}")
print(f"  Máximo: {best_f1s.max():.4f}")
print(f"  Media: {best_f1s.mean():.4f}")
print(f"  Mediana: {np.median(best_f1s):.4f}")
print(f"  Std: {best_f1s.std():.4f}")
print()
print(f"BSPs con F1 > 0.8: {(best_f1s > 0.8).sum()} ({(best_f1s > 0.8).mean()*100:.1f}%)")
print(f"BSPs con F1 > 0.5: {(best_f1s > 0.5).sum()} ({(best_f1s > 0.5).mean()*100:.1f}%)")
print(f"BSPs con F1 < 0.2: {(best_f1s < 0.2).sum()} ({(best_f1s < 0.2).mean()*100:.1f}%)")

```

Distribución de F1 scores:

```

Mínimo: 0.0000
Máximo: 1.0000
Media: 0.7182
Mediana: 1.0000
Std: 0.4332

```


BSPs con F1 > 0.8: 94 (73.4%)
 BSPs con F1 > 0.5: 94 (73.4%)
 BSPs con F1 < 0.2: 34 (26.6%)

```
# Top 10 BSPs mejor detectadas
top_indices = np.argsort(best_f1s)[-10:][::-1]

print("\nTop 10 BSPs mejor detectadas:")
print("="*60)
print(f"{'BSP':<10} {'F1':<8} {'Feature':<10} {'Threshold':<10}")
print("-"*60)
for idx in top_indices:
    bsp_name = bsp_pieces_names[idx]
    f1 = best_f1s[idx]
    feat = best_features[idx]
    thresh = best_thresholds[idx]
    print(f"{bsp_name:<10} {f1:<8.4f} {feat:<10} {thresh:<10.1f}")
```

Top 10 BSPs mejor detectadas:

```
=====
```

BSP	F1	Feature	Threshold
BSPH8W	1.0000	3475	0.0
BSPH6B	1.0000	3475	0.0
BSPH5B	1.0000	3475	0.0
BSPH4W	1.0000	3475	0.0
BSPH2B	1.0000	1475	0.2
BSPH1W	1.0000	3475	0.0
BSPG7W	1.0000	3475	0.0
BSPG8W	1.0000	3475	0.0
BSPG4W	1.0000	940	0.0
BSPG4B	1.0000	4137	0.0

```
# Bottom 10 BSPs peor detectadas
bottom_indices = np.argsort(best_f1s)[:10]

print("\nBottom 10 BSPs peor detectadas:")
print("="*60)
print(f"{'BSP':<10} {'F1':<8} {'Feature':<10} {'Threshold':<10}")
print("-"*60)
for idx in bottom_indices:
    bsp_name = bsp_pieces_names[idx]
    f1 = best_f1s[idx]
    feat = best_features[idx]
    thresh = best_thresholds[idx]
    print(f"{bsp_name:<10} {f1:<8.4f} {feat:<10} {thresh:<10.1f}")
```

Bottom 10 BSPs peor detectadas:

```
=====
```

BSP	F1	Feature	Threshold
BSPA1W	0.0000	-1	-1.0
BSPA2W	0.0000	-1	-1.0
BSPA3B	0.0000	-1	-1.0
BSPA8W	0.0000	-1	-1.0
BSPA7B	0.0000	-1	-1.0
BSPB8W	0.0000	-1	-1.0
BSPD7B	0.0000	-1	-1.0

BSPC8W	0.0000	-1	-1.0
BSPD1W	0.0000	-1	-1.0
BSPD8B	0.0000	-1	-1.0

```
# Distribución de thresholds óptimos
unique_thresholds, counts = np.unique(best_thresholds, return_counts=True)

print("\nDistribución de thresholds óptimos:")
print("="*60)
for t, count in zip(unique_thresholds, counts):
    if t >= 0: # Ignorar -1 (BSPs sin match)
        percentage = count / len(best_thresholds) * 100
        print(f"Threshold {t:.1f}: {count:3d} BSPs ({percentage:.1f}%)"
```

Distribución de thresholds óptimos:

```
=====
Threshold 0.0: 70 BSPs (54.7%)
Threshold 0.1: 12 BSPs (9.4%)
Threshold 0.2: 8 BSPs (6.2%)
Threshold 0.3: 1 BSPs (0.8%)
Threshold 0.4: 2 BSPs (1.6%)
Threshold 0.7: 1 BSPs (0.8%)
```

```
# =====
# CONSULTA POR BSP ESPECÍFICA
# =====

# >>> CAMBIA AQUÍ LA BSP QUE QUIERES ANALIZAR <
BSP_OBJETIVO = "BSPH8W"

# Buscar índice de la BSP
bsp_idx = bsp_piezas_names.index(BSP_OBJETIVO)

# Extraer resultados ya calculados
neurona = best_features[bsp_idx]
threshold = best_thresholds[bsp_idx]
f1 = best_f1s[bsp_idx]

# f_max de la neurona ganadora (sae_features ya está en memoria)
f_max_neurona = sae_features[:, neurona].max().item()

# Activaciones de esa neurona en todos los tableros
activaciones_neurona = sae_features[:, neurona].cpu().numpy()

print(f"BSP analizada: {BSP_OBJETIVO}")
print(f"Neurona ganadora: {neurona}")
print(f"Threshold óptimo: {threshold:.1f}")
print(f"F_max neurona: {f_max_neurona:.4f}")
print(f"Umbral efectivo: {threshold:.1f} x {f_max_neurona:.4f} = {threshold * f_max_neurona:.4f}")
print(f"F1 score: {f1:.4f}")
print(f"\nActivaciones de neurona {neurona} ({len(activaciones_neurona)} tableros):")
for i, act in enumerate(activaciones_neurona):
    print(f" Tablero {i+1:5d}: {act:.6f}")
```

```
BSP analizada: BSPH8W
Neurona ganadora: 3475
Threshold óptimo: 0.0
F_max neurona: 0.6508
Umbral efectivo: 0.0 x 0.6508 = 0.0000
F1 score: 1.0000
```

Activaciones de neurona 3475 (15 tableros):

```
Tablero    1: 0.353695
Tablero    2: 0.580835
Tablero    3: 0.035597
Tablero    4: 0.340539
Tablero    5: 0.024847
Tablero    6: 0.577339
Tablero    7: 0.288560
Tablero    8: 0.068226
Tablero    9: 0.256513
Tablero   10: 0.411264
Tablero   11: 0.194918
Tablero   12: 0.047416
Tablero   13: 0.650777
Tablero   14: 0.029245
Tablero   15: 0.273099
```

```
def plot_bsp_histogram(BSP_HISTOGRAMA, sae_features, bsp_pieces_names, bsp_pieces_gt_tensor,
                       best_features, best_thresholds, best_fls, n_bins=20):

    # Extraer datos de la BSP
    bsp_idx_hist = bsp_pieces_names.index(BSP_HISTOGRAMA)
    neurona_hist = best_features[bsp_idx_hist]
    threshold_hist = best_thresholds[bsp_idx_hist]
    f1_hist = best_fls[bsp_idx_hist]
    f_max_hist = sae_features[:, neurona_hist].max().item()

    # Activaciones y ground truth
    activaciones = sae_features[:, neurona_hist].cpu().numpy()
    gt = bsp_pieces_gt_tensor[:, bsp_idx_hist].cpu().numpy()
    umbral = threshold_hist * f_max_hist

    # Clasificar
    predicho = (activaciones > umbral).astype(int)
    tp_mask = (gt == 1) & (predicho == 1)
    fp_mask = (gt == 0) & (predicho == 1)
    tn_mask = (gt == 0) & (predicho == 0)
    fn_mask = (gt == 1) & (predicho == 0)

    # Bins
    bins = np.linspace(0, activaciones.max(), n_bins + 1)
    bin_centers = (bins[:-1] + bins[1:]) / 2
    bin_width = bins[1] - bins[0]

    tp_counts, _ = np.histogram(activaciones[tp_mask], bins=bins)
    fp_counts, _ = np.histogram(activaciones[fp_mask], bins=bins)
    tn_counts, _ = np.histogram(activaciones[tn_mask], bins=bins)
    fn_counts, _ = np.histogram(activaciones[fn_mask], bins=bins)

    left_mask = bin_centers <= umbral
    right_mask = bin_centers > umbral

    # Graficar
    fig, ax = plt.subplots(figsize=(14, 6))

    ax.bar(bin_centers[left_mask], tn_counts[left_mask],
           width=bin_width, color='steelblue', alpha=0.9,
           label='TN (GT=0, pred=0)', align='center')
    ax.bar(bin_centers[left_mask], fn_counts[left_mask],
           width=bin_width, bottom=tn_counts[left_mask],
           color='lightcoral', alpha=0.9,
           label='FN (GT=1, pred=0)', align='center')
```

```

ax.bar(bin_centers[right_mask], tp_counts[right_mask],
       width=bin_width, color='green', alpha=0.9,
       label='TP (GT=1, pred=1)', align='center')
ax.bar(bin_centers[right_mask], fp_counts[right_mask],
       width=bin_width, bottom=tp_counts[right_mask],
       color='orange', alpha=0.9,
       label='FP (GT=0, pred=1)', align='center')

# Números TN
for i, x in enumerate(bin_centers[left_mask]):
    total = tn_counts[left_mask][i] + fn_counts[left_mask][i]
    if tn_counts[left_mask][i] > 0:
        ax.text(x, total + 0.3, str(tn_counts[left_mask][i]),
                ha='center', va='bottom', fontsize=7, color='steelblue')

# Números TP
for i, x in enumerate(bin_centers[right_mask]):
    total = tp_counts[right_mask][i] + fp_counts[right_mask][i]
    if tp_counts[right_mask][i] > 0:
        ax.text(x, total + 0.3, str(tp_counts[right_mask][i]),
                ha='center', va='bottom', fontsize=7, color='green')

# Umbral
ax.axvline(x=umbral, color='red', linewidth=2, linestyle='--',
          label=f'Umbral = {threshold_hist:.1f} × {f_max_hist:.3f} = {umbral:.4f}')

ax.set_xlabel('Activación de la neurona')
ax.set_ylabel('Conteo de tableros')
ax.set_title(f'Histograma BSP: {BSP_HISTOGRAMA} | Neurona: {neurona_hist} | F1: {f1_hist:.4f}')
ax.legend()
ax.text(umbral * 0.5, ax.get_ylim()[1] * 0.95, 'predicho = 0',
       ha='center', fontsize=10, color='gray')
ax.text(umbral + (activaciones.max() - umbral) * 0.5, ax.get_ylim()[1] * 0.95, 'predicho = 1',
       ha='center', fontsize=10, color='gray')

plt.tight_layout()
plt.show()

print(f"\nResumen:")
print(f"  TP: {tp_mask.sum()} | FP: {fp_mask.sum()}")
print(f"  TN: {tn_mask.sum()} | FN: {fn_mask.sum()}")
print(f"  F1: {f1_hist:.4f}")

print(" Función plot_bsp_histogram definida")

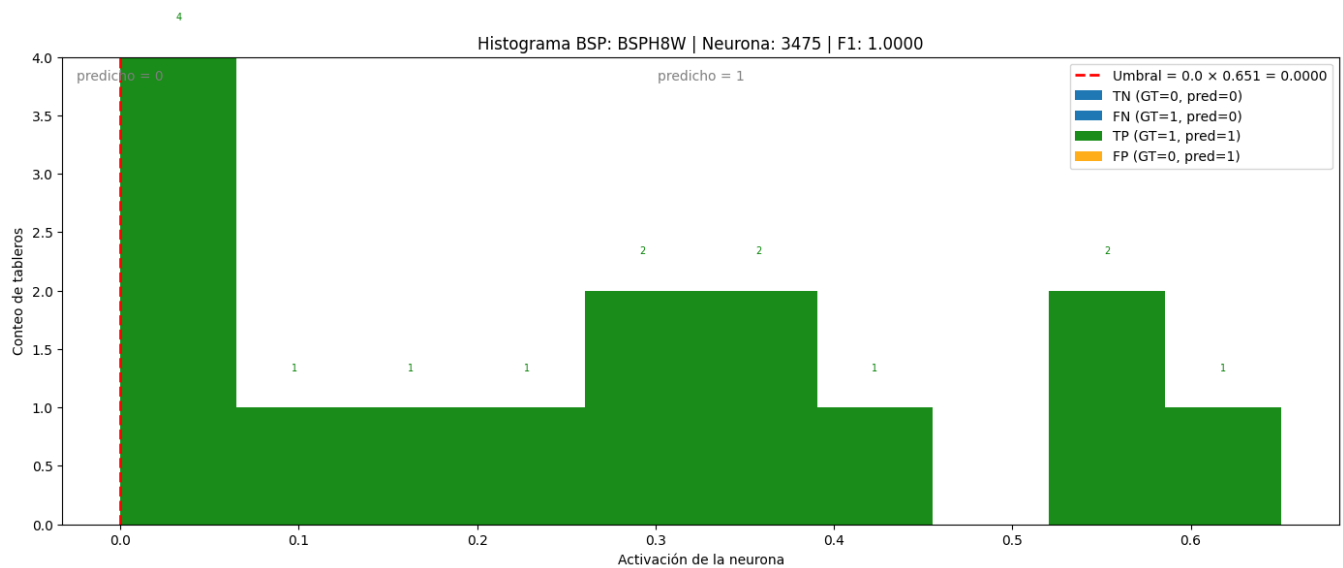
```

Función plot_bsp_histogram definida

```

# >>> CAMBIA AQUÍ LA BSP QUE QUIERES VISUALIZAR <
import matplotlib.pyplot as plt
plot_bsp_histogram(
    BSP_HISTOGRAMA      = "BSPH8W",
    sae_features         = sae_features,
    bsp_pieces_names    = bsp_pieces_names,
    bsp_pieces_gt_tensor = bsp_pieces_gt_tensor,
    best_features        = best_features,
    best_thresholds      = best_thresholds,
    best_f1s             = best_f1s,
    n_bins               = 10
)

```



Resumen:

TP: 15 | FP: 0
 TN: 0 | FN: 0
 F1: 1.0000

0.8 8. Guardar Resultados

```
# Guardar resultados en el directorio de métricas del experimento
results = {
    'coverage': coverage,
    'best_f1s': best_f1s,
    'best_features': best_features,
    'best_thresholds': best_thresholds,
    'bsp_names': bsp_pieces_names,
    'execution_time_seconds': elapsed_time
}

metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)
output_path = metrics_dir / f"coverage_{filter_suffix}.npz"
np.savez(output_path, **results)

print(f" Resultados guardados en:")
print(f"  {output_path}")
```

Resultados guardados en:

c:\Users\Esposa\Documents\Repos\othello_world\experiments\layer_06\sae_standard\sae_l1-decay_standard\200games\ex1\me

0.9 9. Generar PDF con Quarto

```

import subprocess

# Guardar PDF en el mismo directorio que el .npz (metrics/)
pdf_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    f'coverage_{filter_suffix}',
    extras_id=extras_id
)

notebook_name = 'coverage_gpu_optimized.ipynb'
output_path = pdf_dir / pdf_name

print(f' Generando PDF con Quarto...')
print(f' Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{pdf_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f' PDF generado exitosamente: {output_path}')
else:
    print(f' Error al generar PDF:')
    print(result.stderr)

```

```

Generando PDF con Quarto...
Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\experiments\layer_06\sae_standard\sae_l1-
cosannealing_standard\200games\ex1\metrics\sae_standard_l1-cosannealing_l6_200g_ex1_coverage_absolute.pdf
PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\experiments\layer_06\sae_standard\sae_l1-
cosannealing_standard\200games\ex1\metrics\sae_standard_l1-cosannealing_l6_200g_ex1_coverage_absolute.pdf

```